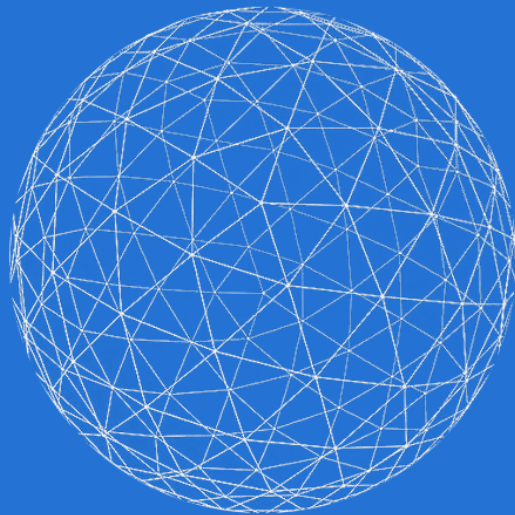




第二章 自动驾驶开发环境搭建





CONTENTS

目录

- 1 ROS 系统简介
- 2 自动驾驶汽车所需的软件环境配置



教学目标和重难点

➤ 教学目标

1. 熟悉ROS系统和Ubuntu系统

重点

2. 掌握自动驾驶汽车所需的环境的搭建

重点



① ROS 系统简介



ROS 系统简介

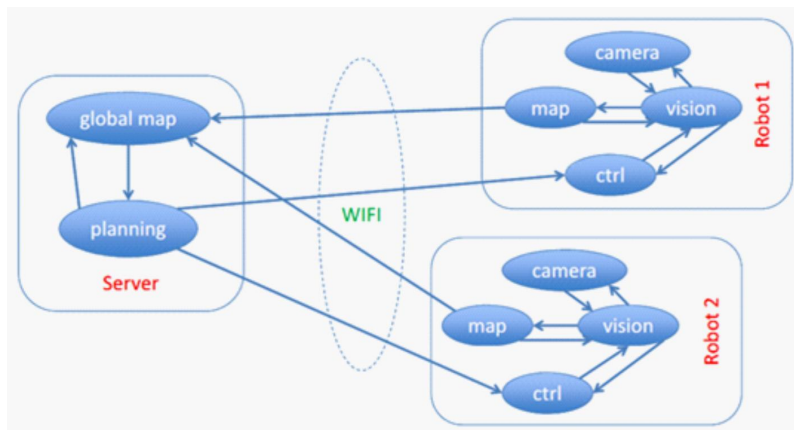
- ROS为Robot Operating System（机器人操作系统）的简写，是一个面向机器人的开源元操作系统（open-source, meta-operating system）。
- 它能够提供类似传统操作系统的诸多功能，如硬件抽象、底层设备控制、进程间消息传递和程序包管理等。此外，它还提供相关工具和库，用于获取、编译、编辑代码以及在多个计算机之间运行程序完成分布式计算。
- ROS的运行架构是一种使用ROS通信模块实现模块间P2P的松耦合的网络连接的处理架构，它执行若干种类型的通讯，包括基于服务的同步RPC（远程过程调用）通讯、基于Topic的异步数据流通讯，还有参数服务器上的数据存储。



ROS 系统简介

- ROS的主要目标是为机器人研究和开发提供代码复用的支持。
- ROS是一个分布式的进程（即“节点”）框架，这些进程被封装在易于被分享和发布的程序包和功能包中。
- ROS也支持一种类似于代码储存库的联合系统，这个系统也可以实现工程的协作及发布。可以使一个工程的开发和实现从文件系统到用户接口完全独立决策（不受ROS限制）。同时，所有的工程都可以被ROS的基础工具整合在一起。
- ROS在高层架构上提供了更多的算法应用（如集成OpenCV的视觉算法）。

➤ 特点-点对点设计



ROS通过点对点设计以及服务和节点管理等机制可以分散由于计算机视觉和语音识别等功能带来的实时计算压力，这种设计能适应服务机器人遇到的调战。

➤ 特点-不依赖编程语言

ROS支持多种现代高级编程语言，C++、Python和Lisp语言已经在ROS中实现编译，并得到应用，Octave和Java的测试库也已经实现。为了支持多语言编程，ROS采用了一种语言中立的接口定义语言（language-neutral interface definition language,IDL）来实现各模块之间的消息传送。



➤ 特点-精简与集成

ROS建立的系统具有模块化的特点，各模块中的代码可以单独编译，而且编译使用的CMake工具使它很容易的就实现精简的理念。ROS基本将复杂的代码封装在库里，只是创建了一些小的应用程序为ROS显示库的功能，这就允许了对简单的代码超越原型进行移植和重新使用。作为一种新加入的有优势，单元测试当代码在库中分散后也变得非常的容易，一个单独的测试程序可以测试库中很多的特点。

ROS不修改用户的主函数，所以代码可以被其他的机器人软件使用。其优点是ROS很容易和其他的机器人软件平台集成。例如，在计算机视觉方面，ROS已经与OpenCV实现集成。在驱动、导航和模拟器方面，ROS已经与Player系统实现集成。在规划算法方面，ROS也已与OpenAVE系统实现集成。

➤ 特点-便于测试

为机器人开发软件比其他软件开发更具挑战性，主要是因为调试准备时间长，且调试过程复杂。况且，因为硬件维修、经费有限等因素，不一定随时有机器人可供使用。ROS提供两种策略来解决上述问题。

- ◆ 精心设计的ROS系统框架将底层硬件控制模块和顶层数据处理与决策模块分离，从而可以使用模拟器替代底层硬件模块，独立测试顶层部分，提高测试效率。
- ◆ ROS另外提供了一种简单的方法可以在调试过程中记录传感器数据及其他类型的消息数据，并在试验后按时间戳回放。通过这种方式，每次运行机器人可以获得更多的测试机会。例如，可以记录传感器的数据，并通过多次回放测试不同的数据处理算法

➤ 特点-开源

ROS遵从BSD协议，这给了使用者很大的自由，使开发者可以清楚的查看、自由的使用源代码，如果有需要，可以根据不同的系统及硬件环境对源代码进行修改，或者进行二次开发。



➤ 特点-强大的库及社区

ROS提供了广泛的库文件实现以机动性、操作控制、感知为主的机器人功能。同时由于其开源特性，ROS的支持与发展依托着一个强大的社区。其官方网站尤其关注兼容性和支持文档，提供了一套 “一站式” 的方案使得用户得以搜索并学习来自全球开发者数以千计的ROS程序包。



② 自动驾驶汽车所需的软件环境配置



自动驾驶汽车所需的软件环境配置

自动驾驶汽车所需的软件如下：

- Ubuntu系统
- ROS系统



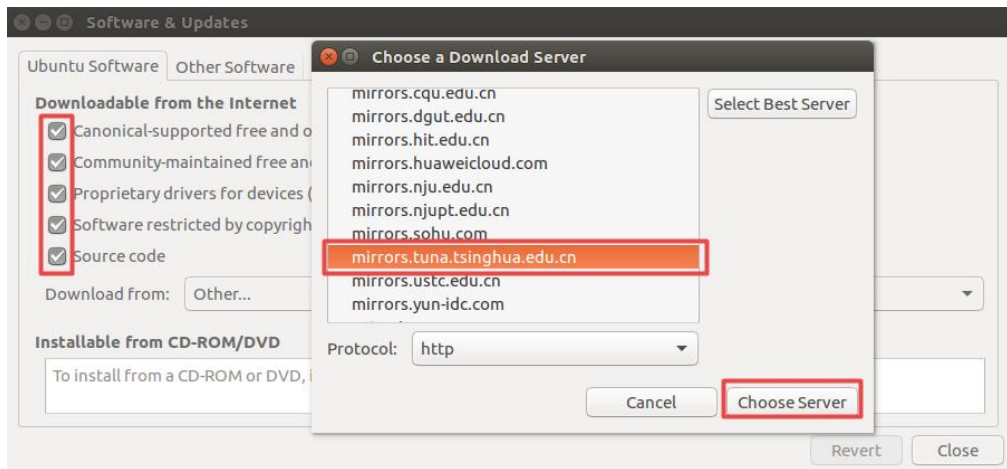
 ROS



自动驾驶汽车所需的软件环境配置

➤ 2.1 更改源，将下载源更改为清华源

在Ubuntu设置--Software & Update中设置如下：

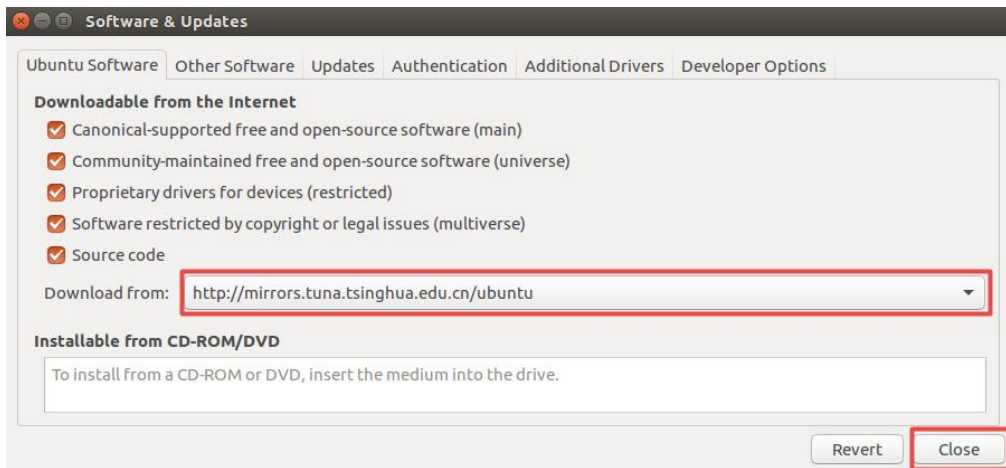




自动驾驶汽车所需的软件环境配置

➤ 2.1 更改源，将下载源更改为清华源

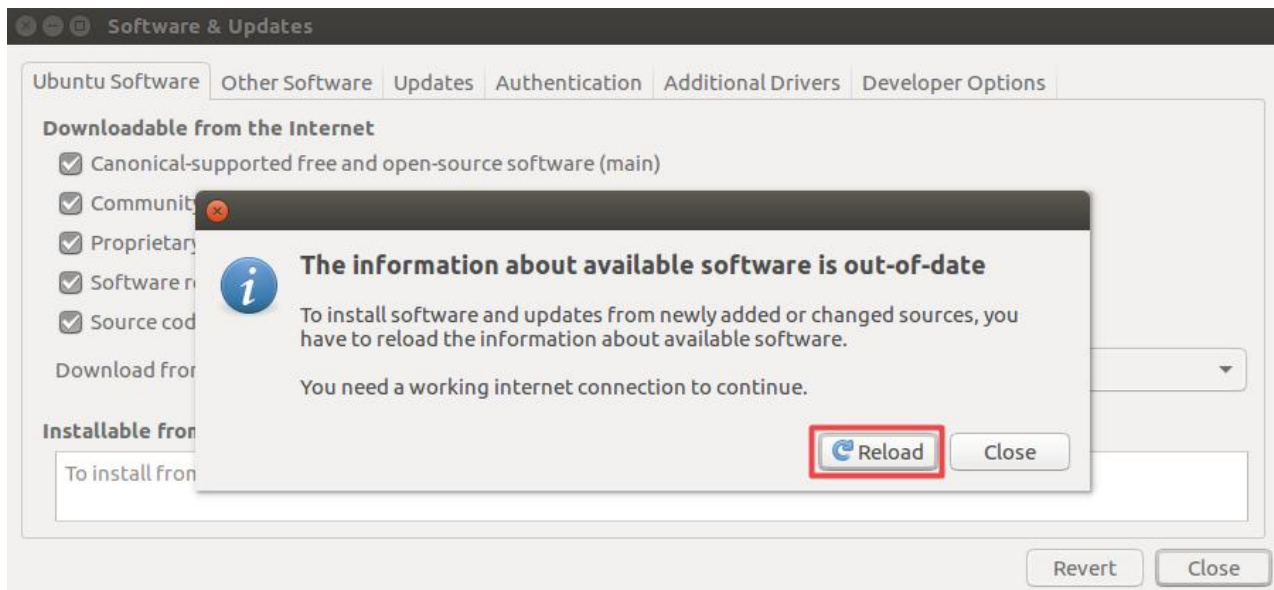
单击“选择服务器”后输入用户密码，点击“授权”。关闭页面。



自动驾驶汽车所需的软件环境配置

➤ 2.1 更改源，将下载源更改为清华源

单击“重新加载”，等待更新完毕即可。





自动驾驶汽车所需的软件环境配置

➤ 2.2 设置 sources.list

单击“重新加载”，等待更新完毕即可。

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc)
main" > /etc/apt/sources.list.d/ros-latest.list'
```



自动驾驶汽车所需的软件环境配置

➤ 2.3 添加秘钥

```
sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 --recv-key  
C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
```

```
uniwise@uniwise:~$ sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 --  
recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654  
Executing: /tmp/tmp.eeuwou9rna/gpg.1.sh --keyserver  
hkp://keyserver.ubuntu.com:80  
--recv-key  
C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654  
gpg: requesting key AB17C654 from hkp server keyserver.ubuntu.com  
gpg: key AB17C654: public key "Open Robotics <info@osrfoundation.org>" imported  
gpg: Total number processed: 1  
gpg: imported: 1 (RSA: 1)  
uniwise@uniwise:~$
```



自动驾驶汽车所需的软件环境配置

➤ 2.4 更新package

```
sudo apt-get update  
sudo apt-get upgrade
```

➤ 2.5 安装ROS kinetic完整版

sudo apt-get install ros-kinetic-desktop-full

```
Setting up ros-noetic-rviz-plugin-tutorials (0.11.0-1focal.20221209.231034) ...
Setting up ros-noetic-rqt-reconfigure (0.5.5-1focal.20221209.205649) ...
Setting up ros-noetic-rqt-robot-dashboard (0.5.8-1focal.20221209.214118) ...
Setting up ros-noetic-visualization-tutorials (0.11.0-1focal.20221209.231532) ..
.
Setting up ros-noetic-ros-core (1.5.0-1focal.20221124.043914) ...
Setting up ros-noetic-geometry (1.13.2-1focal.20221209.221400) ...
Setting up ros-noetic-rqt-srv (0.4.9-1focal.20221209.214249) ...
Setting up ros-noetic-rqt-action (0.4.9-1focal.20221209.205757) ...
Setting up ros-noetic-urdf-sim-tutorial (0.5.1-1focal.20221209.223812) ...
Setting up ros-noetic-gazebo-plugins (2.9.2-1focal.20221209.220915) ...
Setting up ros-noetic-rqt-launch (0.4.9-1focal.20221209.205644) ...
Setting up ros-noetic-gazebo-ros-pkgs (2.9.2-1focal.20221209.230134) ...
Setting up ros-noetic-laser-pipeline (1.6.4-1focal.20221209.224944) ...
Setting up ros-noetic-ros-base (1.5.0-1focal.20221209.203123) ...
Setting up ros-noetic-rqt-robot-plugins (0.5.8-1focal.20221209.231519) ...
Setting up ros-noetic-robot (1.5.0-1focal.20221209.225949) ...
Setting up ros-noetic-rqt-common-plugins (0.4.9-1focal.20221209.214337) ...
Setting up ros-noetic-perception (1.5.0-1focal.20221209.225046) ...
Setting up ros-noetic-viz (1.5.0-1focal.20221209.231555) ...
Setting up ros-noetic-desktop (1.5.0-1focal.20221209.231659) ...
Setting up ros-noetic-simulators (1.5.0-1focal.20221209.231603) ...
Setting up ros-noetic-desktop-full (1.5.0-1focal.20221209.231810) ...
Processing triggers for libc-bin (2.31-0ubuntu9.9) ...
```



自动驾驶汽车所需的软件环境配置

➤ 2.6 安装辅助包

```
sudo apt install python3-rosdep2
```

➤ 2.7 初始化rosdep

```
sudo rosdep init  
rosdep update
```

➤ 2.7 初始化rosdep

在初始化rosdep的时候，无法访问对应的域名，可以按照以下步骤进行：

1. 修改/usr/lib/python3/dist-packages/rosdep2/sources_list.py，加入代码

`url="https://ghproxy.com/" + url`

```
def download_rosdep_data(url):  
    """  
    :raises: :exc:`DownloadFailure` If data cannot be  
        retrieved (e.g. 404, bad YAML format, server down).  
    """  
    try:  
        url = 'https://ghproxy.com/' + url  
        f = urlopen(url, timeout=DOWNLOAD_TIMEOUT)  
        text = f.read()  
        f.close()  
        data = yaml.safe_load(text)  
        if type(data) != dict:  
            raise DownloadFailure('rosdep data from [%s] is not a YAML dictionary' % (url))  
        return data  
    except (URLError, httpplib.HTTPException) as e:  
        raise DownloadFailure(str(e) + ' (%s)' % url)  
    except yaml.YAMLError as e:  
        raise DownloadFailure(str(e))
```

➤ 2.7 初始化roscdep

2. 将/usr/lib/python3/dist-packages/rosdistro/__init__.py里面的DEFAULT_INDEX_URL 进行如下替换: **DEFAULT_INDEX_URL =**
'https://ghproxy.com/https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml'

```
__version__ = '0.9.0'

# index information

#DEFAULT_INDEX_URL = 'https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml'
DEFAULT_INDEX_URL = 'https://ghproxy.com/https://raw.githubusercontent.com/ros/rosdistro/-
master/index-v4.yaml'

def get_index_url():
    # environment variable has precedence over configuration files
    if 'ROSDISTRO_INDEX_URL' in os.environ:
        return os.environ['ROSDISTRO_INDEX_URL']
```

➤ 2.7 初始化rosdep

3. 在/usr/lib/python3/dist-packages/rosdep2/gbpdistro_support.py 第204行添加如下代码:

`gbpdistro_url = "https://ghproxy.com/" + gbpdistro_url`

```
# we can convert a gbpdistro file into rosdep data by following a
# couple rules
# will output a warning
targets_data = download_targets_data(targets_url=targets_url)
try:
    gbpdistro_url = "https://ghproxy.com/" + gbpdistro_url
    f = urlopen(gbpdistro_url, timeout=DOWNLOAD_TIMEOUT)
    text = f.read()
    f.close()
    gbpdistro_data = yaml.safe_load(text)
    # will output a warning
    return gbprepo_to_rosdep_data(gbpdistro_data,
                                  targets_data,
                                  gbpdistro_url)
```

➤ 2.7 初始化rosdep

4. 在/usr/lib/python3/dist-packages/rosdep2/gbpdistro_support.py中加入
FUERTE_GBPDISTRO_URL= "https://ghproxy.com/" + FUERTE_GBPDISTRO_URL,
DOWNLOAD_TIMEOUT = 15.0 改成 DOWNLOAD_TIMEOUT = 500.0

```
try:
    unicode
except NameError:
    basestring = unicode = str

# location of an example gbpdistro file for reference and testing

FUERTE_GBPDISTRO_URL = 'https://raw.githubusercontent.com/ros/rosdistro/' \
    'master/releases/fuerte.yaml'
FUERTE_GBPDISTRO_URL = "https://ghproxy.com/" + FUERTE_GBPDISTRO_URL

# seconds to wait before aborting download of gbpdistro data
DOWNLOAD_TIMEOUT = 500.0
```



自动驾驶汽车所需的软件环境配置

➤ 2.7 初始化rosdep

5. 依照步骤4, 在/usr/lib/python3/dist-packages/rosdep2/sources_list.py中添加
`DEFAULT_SOURCES_LIST_URL= "https://ghproxy.com/" +DEFAULT_SOURCES_LIST_URL,`
将`DOWNLOAD_TIMEOUT = 15.0`改成 `DOWNLOAD_TIMEOUT = 500.0`
6. 依照步骤4, 在/usr/lib/python3/dist-packages/rosdep2/rep3.py中添加
`EP3_TARGETS_URL= "https://ghproxy.com/" +REP3_TARGETS_URL,` 将
`DOWNLOAD_TIMEOUT = 15.0`改成 `DOWNLOAD_TIMEOUT = 500.0`
7. 在/usr/lib/python3/dist-packages/rosdistro/manifest_provider/github.py中的第68行和119行添加: `url="https://ghproxy.com/"+url`



自动驾驶汽车所需的软件环境配置

➤ 2.7 初始化rosdep

8. 重新执行rosdep update

```
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://ghproxy.com/https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Add distro "foxy"
Skip end-of-life distro "galactic"
Skip end-of-life distro "groovy"
Add distro "humble"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "jade"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "melodic"
Add distro "noetic"
Add distro "rolling"
updated cache in /home/uniwise/.ros/rosdep/sources.cache
```



自动驾驶汽车所需的软件环境配置

➤ 2.8 配置ROS环境

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

➤ 2.9 安装依赖项

```
sudo apt install python3-rosinstall python3-rosinstall-generator  
python3-wstool
```

➤ 2.10 测试ROS是否安装成功

(1) CTRL+ALT+T打开终端，输入以下命令，初始化ROS环境：

roscore

```
uniwise@uniwise-virtual-machine:~$ roscore
... logging to /home/uniwise/.ros/log/e0623b42-b0c7-11ed-8399-072dcf2e3b75/roslaunch-uniwise-virtual-machine-100168.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://uniwise-virtual-machine:42709/
ros_comm version 1.15.15

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.15

NODES

auto-starting new master
process[master]: started with pid [100176]
ROS_MASTER_URI=http://uniwise-virtual-machine:11311/

setting /run_id to e0623b42-b0c7-11ed-8399-072dcf2e3b75
process[rosout-1]: started with pid [100186]
started core service [/rosout]
```



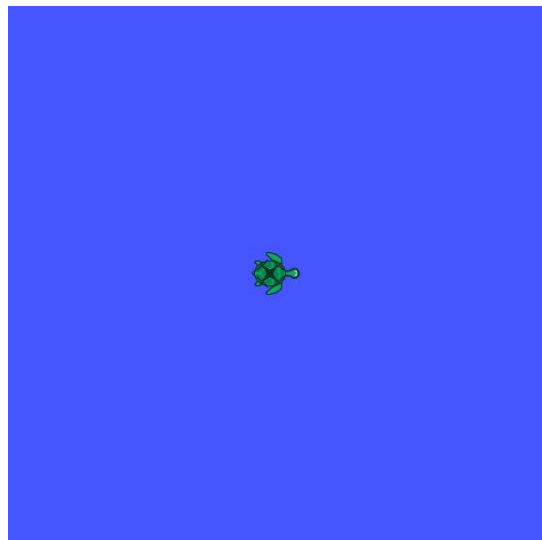
自动驾驶汽车所需的软件环境配置

➤ 2.10 测试ROS是否安装成功

(2) 打开一个新的终端，输入以下命令，弹出一个小乌龟窗口：

```
roslaunch turtlesim turtlesim_node
```

注意：在执行 (2) ~ (4) 步骤的时候，不要关闭前面的终端。





自动驾驶汽车所需的软件环境配置

➤ 2.10 测试ROS是否安装成功

(3) 打开一个新的终端，输入以下命令，用键盘方向键控制小乌龟移动：

```
roslaunch turtlesim turtlesim_key
```

注意：在执行 (2) ~ (4) 步骤的时候，不要关闭前面的终端。

```
uniwise@uniwise-virtual-machine:~$ roslaunch turtlesim turtlesim_key
Reading from keyboard
-----
Use arrow keys to move the turtle. 'q' to quit.
█
```



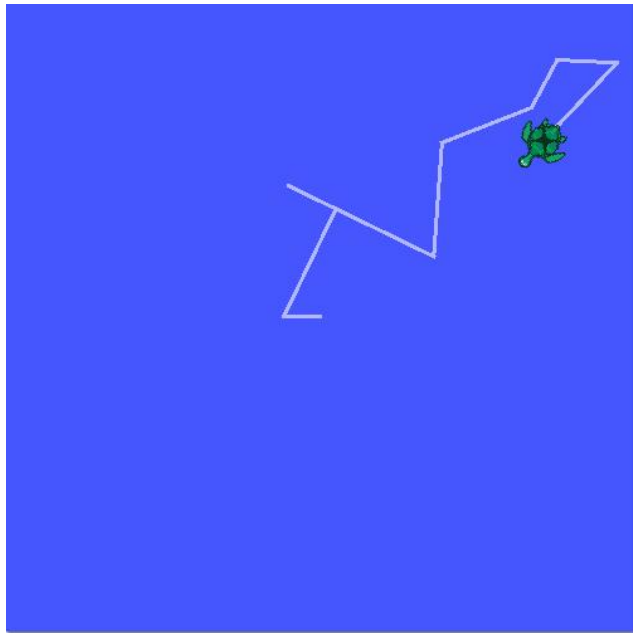
自动驾驶汽车所需的软件环境配置

➤ 2.10 测试ROS是否安装成功

(3) 打开一个新的终端，输入以下命令，用键盘方向键控制小乌龟移动：

可以通过键盘方向键移动小乌龟。

注意：以上命令窗口和小乌龟窗口并列，激活命令窗口，按方向键，可以发现小乌龟在移动。如果激活的是小乌龟窗口，按方向键小乌龟是不动的。





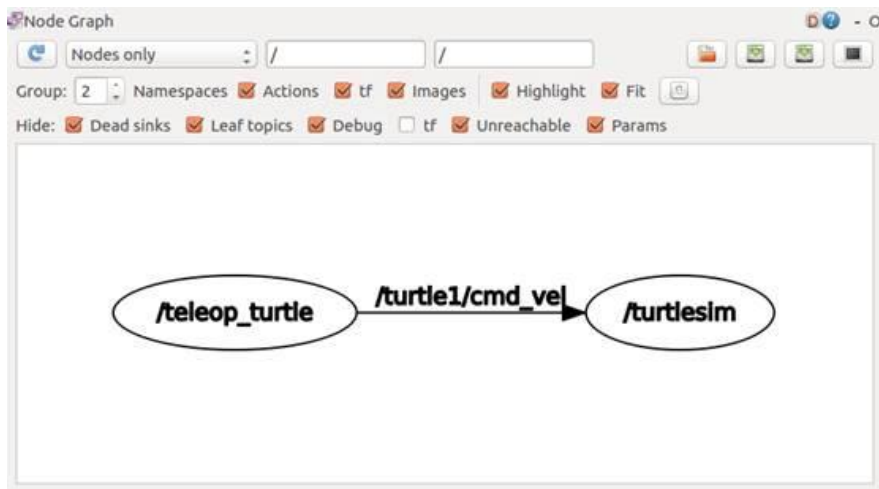
自动驾驶汽车所需的软件环境配置

➤ 2.10 测试ROS是否安装成功

(4) 打开新的Termial，输入以下命令，可以查看ROS节点信息：

```
roslaunch rqt_graph rqt_graph
```

至此，ROS安装成功。



- ROS为Robot Operating System（机器人操作系统）的简写，是一个面向机器人的开源元操作系统。
- 主要目标是为机器人研究和开发提供代码复用的支持。
- ROS也支持一种类似于代码储存库的联合系统，所有的工程都可以被ROS的基础工具整合在一起。
- ROS在高层架构上提供了更多的算法应用（如集成OpenCV的视觉算法）。

➤ ROS的特点:

- 点对点设计
- 不依赖编程语言
- 精简与集成
- 便于测试
- 开源
- 强大的库和社区

➤ 自动驾驶汽车所需的软件如下:

- Ubuntu系统
- ROS系统



Thank You

AI改变生活